



南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch22 视图

Software Institute
Nanjing University
Bei Jia

视图

● 视图的特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不存放视图对应的数据
- 基表中的数据发生变化，从视图中查询出的数据也随之改变



定义视图-建立视图 1

- 语句格式

CREATE VIEW

<视图名> [(<列名> [,<列名>]...)]

AS <子查询>

[WITH CHECK OPTION];

- WITH CHECK OPTION

- 对视图进行UPDATE, INSERT和DELETE操作时要保证更新、插入或删除的行满足视图定义中的谓词条件（即子查询中的条件表达式）

- 子查询可以是任意的SELECT语句，是否可以含有ORDER BY子句和DISTINCT短语，则决定具体系统的实现。



定义视图-建立视图 2

- 组成视图的属性列名：全部省略或全部指定
 - 全部省略：
 - ◆ 由子查询中SELECT目标列中的诸字段组成
 - 明确指定视图的所有列名：
 - ◆ 某个目标列是聚集函数或列表达式
 - ◆ 多表连接时选出了几个同名列作为视图的字段
 - ◆ 需要在视图中为某个列启用新的更合适的名字



定义视图-建立视图 3

- 关系数据库管理系统执行CREATE VIEW语句时只是把视图定义存入数据字典，并不执行其中的SELECT语句。
- 在对视图查询时，按视图的定义从基本表中将数据查出。
- [例3.84] 建立信息系学生的视图。

```
CREATE VIEW IS_Student  
AS  
  
SELECT Sno,Sname,Sage  
FROM Student  
WHERE Sdept= 'IS';
```



定义视图-建立视图 4

- [例3.85]建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生。

```
CREATE VIEW IS_Student  
AS
```

```
SELECT Sno,Sname,Sage  
FROM Student
```

```
WHERE Sdept= 'IS'
```

```
WITH CHECK OPTION;
```

- 定义IS_Student视图时加上了WITH CHECK OPTION子句，对该视图进行插入、修改和删除操作时，RDBMS会自动加上Sdept='IS'的条件。
- 若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，我们称这类视图为行列子集视图。
 - IS_Student视图就是一个行列子集视图。



定义视图-建立视图 5

- 基于多个基表的视图
- [例3.86] 建立信息系选修了1号课程的学生视图（包括学号、姓名、成绩）。

```
CREATE VIEW IS_S1(Sno,Sname,Grade)
```

```
AS
```

```
SELECT Student.Sno,Sname,Grade
```

```
FROM Student,SC
```

```
WHERE Sdept= 'IS' AND
```

```
Student.Sno=SC.Sno AND
```

```
SC.Cno= '1';
```



定义视图-建立视图 6

- 基于视图的视图
- [例3.87] 建立信息系选修了1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_S2
AS
SELECT
Sno,Sname,Grade
FROM IS_S1
WHERE Grade>=90;
```

- 带表达式的视图
- [例3.88] 定义一个反映学生出生年份的视图。

```
CREATE VIEW
BT_S(Sno,Sname,Sbirth)
AS
SELECT
Sno,Sname,2014-Sage
FROM Student;
```



定义视图-建立视图 7

- 分组视图
- [例3.89] 将学生的学号及平均成绩定义为一个视图

```
CREAT VIEW  
S_G(Sno,Gavg)  
AS  
SELECT  
Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno;
```

- [例3.90]将Student表中所有女生记录定义为一个视图

```
CREATE VIEW  
F_Student(F_Sno,name,sex,age,dept)  
AS  
SELECT * /*不指定属性列*/  
FROM Student  
WHERE Ssex='女';
```

- 缺点:
 - 修改基表Student的结构后, Student表与F_Student视图 的映象关系被破坏, 导致该视图不能正确工作。



定义视图-删除视图 1

- 语句的格式:

`DROP VIEW <视图名>[CASCADE];`

- 该语句从数据字典中删除指定的视图定义
- 如果该视图上还导出了其他视图，使用 CASCADE级联删除语句，把该视图和由它导出的所有视图一起删除
- 删除基表时，由该基表导出的所有视图定义都必须显式地使用DROP VIEW语句删除



定义视图-删除视图 2

- [例3.91] 删除视图BT_S和IS_S1

DROP VIEW BT_S; /*成功执行*/

DROP VIEW IS_S1; /*拒绝执行*/

- 要删除IS_S1，需使用级联删除：

DROP VIEW IS_S1 CASCADE;



查询视图 1

- 用户角度：查询视图与查询基本表相同
- 关系数据库管理系统实现视图查询的方法
 - 视图消解法 (View Resolution)
 - ◆ 进行有效性检查
 - ◆ 转换成等价的对基本表的查询
 - ◆ 执行修正后的查询



查询视图 2

- [例3.92] 在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT Sno,Sage
FROM IS_Student
WHERE Sage<20;
```

- 视图消解转换后的查询语句为：

```
SELECT Sno,Sage
FROM Student
WHERE Sdept= 'IS' AND
Sage<20;
```

- [例3.93] 查询选修了1号课程的

```
SELECT
IS_Student.Sno,Sname
FROM IS_Student,SC
WHERE IS_Student.Sno
=SC.Sno AND SC.Cno= '1';
```



查询视图 3

- 视图消解法的局限

- 有些情况下，视图消解法不能生成正确的查询。

- [例3.94]在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *  
FROM S_G  
WHERE Gavg>=90;
```

- S_G视图的子查询定义：

```
CREATE VIEW S_G (Sno,Gavg)  
AS  
SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno;
```

- 错误：

```
SELECT Sno, AVG(Grade)  
FROM SC  
WHERE AVG(Grade)>=90  
GROUP BY Sno;
```

- 正确：

```
SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno  
HAVING AVG(Grade)>=90;
```

- 或者

```
SELECT *  
FROM (SELECT Sno,AVG(Grade)  
FROM SC  
GROUP BY Sno) AS S_G(Sno,Gavg)  
WHERE Gavg>=90;
```



更新视图 1

- [例3.95] 将信息系学生视图IS_Student中学号“201215122”的学生姓名改为“刘辰”。

```
UPDATE IS_Student  
SET Sname= '刘辰'  
WHERE Sno= ' 201215122 ';
```

- 转换后的语句:

```
UPDATE Student  
SET Sname= '刘辰'  
WHERE Sno= ' 201215122 ' AND Sdept= 'IS';
```



更新视图 2

- [例3.96] 向信息系学生视图IS_S中插入一个新的学生记录，其中学号为“201215129”，姓名为“赵新”，年龄为20岁

```
INSERT
```

```
INTO IS_Student
```

```
VALUES('201215129','赵新',20);
```

- 转换为对基本表的更新:

```
INSERT
```

```
INTO Student(Sno,Sname,Sage,Sdept)
```

```
VALUES('200215129 ','赵新',20,'IS' );
```



更新视图 3

- [例3.97]删除信息系学生视图IS_Student中学号为"201215129"的记录

DELETE

FROM IS_Student

WHERE Sno= ' 201215129 ';

- 转换为对基本表的更新:

DELETE

FROM Student

WHERE Sno= ' 201215129 ' AND Sdept= 'IS';



更新视图 4

- 允许对行列子集视图进行更新
- 对其他类型视图的更新不同系统有不同限制
 - 更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新
- 例：例3.89定义的视图S_G为不可更新视图。

```
UPDATE S_G  
SET   Gavg=90  
WHERE Sno= '201215121';
```
- 这个对视图的更新无法转换成对基本表SC的更新



更新视图 5

- 例：将SC中成绩在平均成绩之上的元组定义成一个视图

```
CREATE VIEW GOOD_SC
```

```
AS
```

```
SELECT Sno,Cno,Grade
```

```
FROM SC
```

```
WHERE Grade >
```

```
(SELECT AVG(Grade)
```

```
FROM SC);
```

- 一个不允许更新的视图上定义的视图也不允许更新



视图的作用-

视图能够简化用户的操作

- 当视图中数据不是直接来自基本表时，定义视图能够简化用户的操作
 - 基于多张表连接形成的视图
 - 基于复杂嵌套查询的视图
 - 含导出属性的视图



视图的作用-

视图使用户能以多种角度看待同一数据

- 视图机制能使不同用户以不同方式看待同一数据,
- 适应数据库共享的需要



视图的作用-

视图对重构数据库提供了一定程度的逻辑独立性

- 例：学生关系Student(Sno,Sname,Ssex,Sage,Sdept)

- “垂直”地分成两个基本表：

SX(Sno,Sname,Sage)

SY(Sno,Ssex,Sdept)

- 通过建立一个视图Student：

```
CREATE VIEW Student(Sno,Sname,Ssex,Sage,Sdept)
```

```
AS
```

```
SELECT SX.Sno,SX.Sname,SY.Ssex,SX.Sage,SY.Sdept
```

```
FROM SX,SY
```

```
WHERE SX.Sno=SY.Sno;
```

- 使用户的外模式保持不变，用户的应用程序通过视图仍然能够查找数据
- 由于对视图的更新是有条件的，因此应用程序中修改数据的语句可能仍会因基本表结构的改变而改变。



视图的作用-

视图能够对机密数据提供安全保护

- 对不同用户定义不同视图，使每个用户只能看到他有权看到的数据



视图的作用-

适当的利用视图可以更清晰的表达查询

- 经常需要执行这样的查询“对每个同学找出他获得最高成绩的课程号”。
- 可以先定义一个视图，求出每个同学获得的最高成绩

```
CREATE VIEW VMGRADE
```

```
AS
```

```
SELECT Sno, MAX(Grade) Mgrade
```

```
FROM SC
```

```
GROUP BY Sno;
```

- 然后用如下的查询语句完成查询：

```
SELECT SC.Sno,Cno
```

```
FROM SC,VMGRADE
```

```
WHERE SC.Sno=VMGRADE.Sno AND
```

```
SC.Grade=VMGRADE .Mgrade;
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch23 数据库安全性

Software Institute
Nanjing University
Bei Jia

数据库安全性

- 问题的提出
 - 数据库的一大特点是数据可以共享
 - 数据共享必然带来数据库的安全性问题
 - 数据库系统中的数据共享不能是无条件的共享
 - ◆ 军事秘密、国家机密、新产品实验数据、市场需求分析、市场营销策略、销售计划、客户档案、医疗档案、银行储蓄数据
- 数据库的安全性是指保护数据库以防止不合法使用所造成的数据泄露、更改或破坏。
- 系统安全保护措施是否有效是数据库系统主要的性能指标之一。



数据库的不安全因素

- 非授权用户对数据库的恶意存取和破坏
 - 一些黑客（Hacker）和犯罪分子在用户存取数据库时猎取用户名和用户口令，然后假冒合法用户偷取、修改甚至破坏用户数据。
 - 数据库管理系统提供的安全措施主要包括用户身份鉴别、存取控制和视图等技术。
- 数据库中重要或敏感的数据被泄露
 - 黑客和敌对分子千方百计盗窃数据库中的重要数据，一些机密信息被暴露。
 - 数据库管理系统提供的主要技术有强制存取控制、数据加密存储和加密传输等。
 - 审计日志分析
- 安全环境的脆弱性
 - 数据库的安全性与计算机系统的安全性紧密联系
 - ◆ 计算机硬件、操作系统、网络系统等的安全性
 - 建立一套可信（Trusted）计算机系统的概念和标准



TCSEC/TDI安全级别划分 1

● TCSEC/TDI安全级别划分

- 按系统可靠或可信程度逐渐增高
- 各安全级别之间具有一种偏序向下兼容的关系

安全级别	定义
A1	验证设计（Verified Design）
B3	安全域（Security Domains）
B2	结构化保护（Structural Protection）
B1	标记安全保护（Labeled Security Protection）
C2	受控的存取保护（Controlled Access Protection）
C1	自主安全保护（Discretionary Security Protection）
D	最小保护（Minimal Protection）



TCSEC/TDI安全级别划分 2

- D级
 - 将一切不符合更高标准的系统均归于D组
- C1级
 - 非常初级的自主安全保护
 - 能够实现对用户和数据的分离，进行自主存取控制（DAC），保护或限制用户权限的传播。
 - 现有的商业系统稍作改进即可满足
- C2级
 - 安全产品的最低档次
 - 提供受控的存取保护，将C1级的DAC进一步细化，以个人身份注册负责，并实施审计和资源隔离
 - 达到C2级的产品在其名称中往往不突出“安全”（Security）这一特色
- B1级
 - 标记安全保护。“安全”（Security）或“可信的”（Trusted）产品。
 - 对系统的数据加以标记，对标记的主体和客体实施强制存取控制（MAC）、审计等安全机制
- B2级
 - 结构化保护
 - 建立形式化的安全策略模型并对系统内的所有主体和客体实施DAC和MAC
- B3级
 - 安全域
 - 该级的TCB必须满足访问监控器的要求，审计跟踪能力更强，并提供系统恢复过程
- A1级
 - 验证设计，即提供B3级保护的同时给出系统的形式化设计说明和验证以确信各安全保护真正实现。



TCSEC/TDI安全级别划分 3

- CC

- 提出国际公认的表述信息技术安全性的结构
- 把信息产品的安全要求分为
 - ◆ 安全功能要求
 - ◆ 安全保证要求

- CC文本组成

- 简介和一般模型
 - ◆ 有关术语、基本概念和一般模型以及与评估有关的一些框架
- 安全功能要求
 - ◆ 列出了一系列类、子类和组件
- 安全保证要求
 - ◆ 列出了一系列保证类、子类和组件
 - ◆ 提出了评估保证级（Evaluation Assurance Level, EAL），从EAL1至EAL7共分为七级



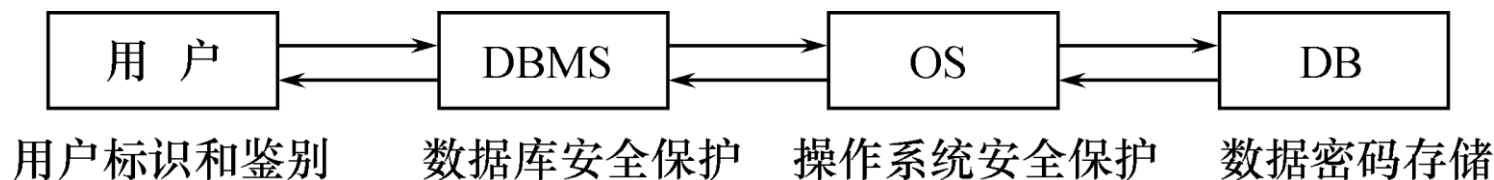
CC评估保证级（EAL）划分

评估保证级	定 义	TCSEC安全级别 (近似相当)
EAL1	功能测试（ functionally tested ）	
EAL2	结构测试（ structurally tested ）	C1
EAL3	系统地测试和检查（ methodically tested and checked ）	C2
EAL4	系统地设计、测试和复查（ methodically designed, tested, and reviewed ）	B1
EAL5	半形式化设计和测试（ semiformally designed and tested ）	B2
EAL6	半形式化验证的设计和测试（ semiformally verified design and tested ）	B3
EAL7	形式化验证的设计和测试（ formally verified design and tested ）	A1

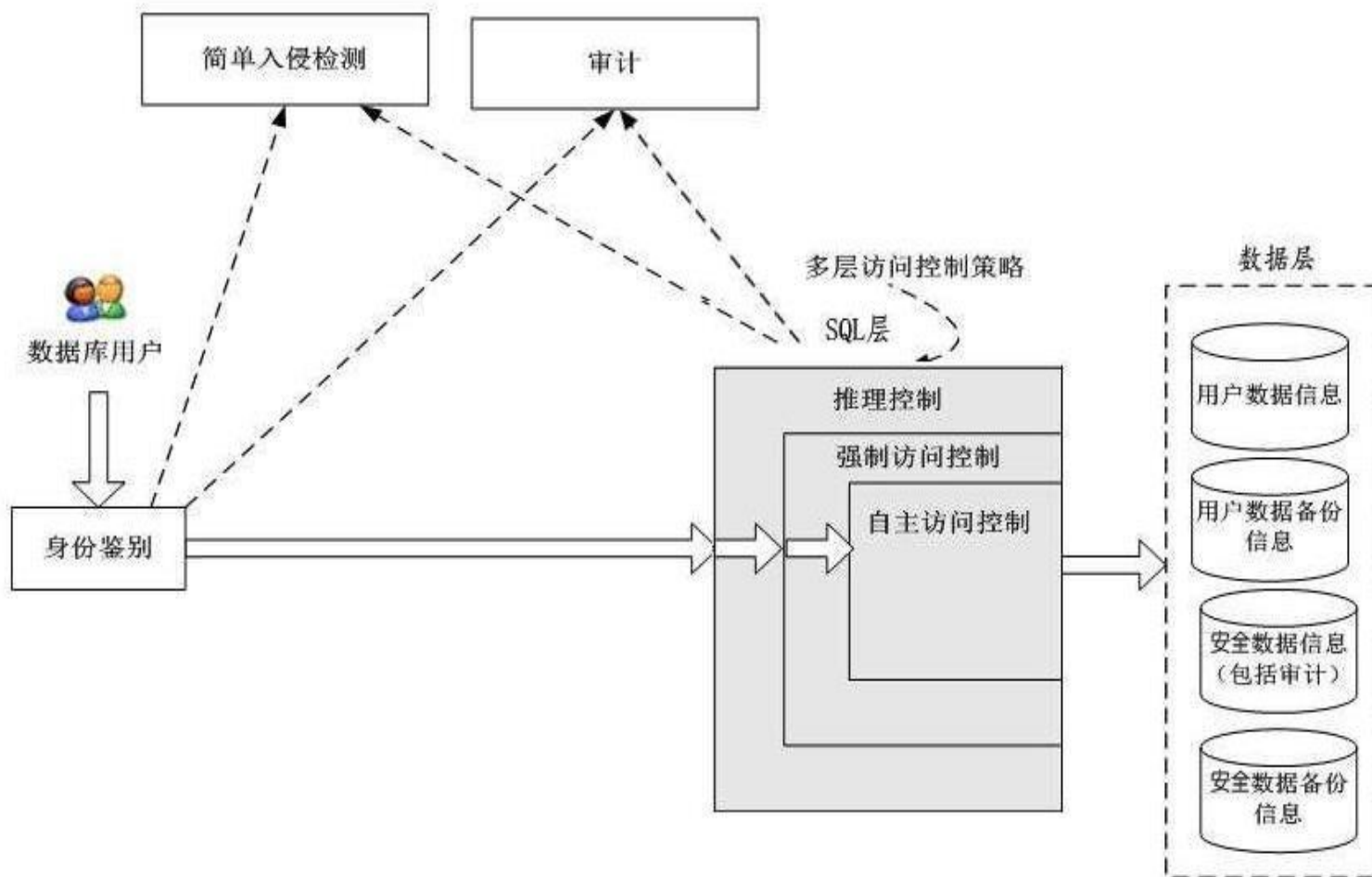


计算机系统的安全模型

- 计算机系统中，安全措施是一级一级层层设置
- 系统根据用户标识鉴定用户身份，合法用户才准许进入计算机系统
- 数据库管理系统还要进行存取控制，只允许用户执行合法操作
- 操作系统有自己的保护措施
- 数据以密码形式存储到数据库中



数据库管理系统安全性控制模型





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch24 存取控制

Software Institute
Nanjing University
Bei Jia

存取控制

- 存取控制机制组成
 - 定义用户权限，并将用户权限登记到数据字典中
 - ◆ 用户对某一数据对象的操作权力称为权限
 - ◆ DBMS提供适当的语言来定义用户权限，存放在数据字典中，称做安全规则或授权规则
 - 合法权限检查
 - ◆ 用户发出存取数据库操作请求
 - ◆ DBMS查找数据字典，进行合法权限检查
- 用户权限定义和合法权检查机制一起组成了数据库管理系统的存取控制子系统

自主存取控制

- 自主存取控制 (Discretionary Access Control, 简称DAC)
 - C2级
 - 用户对不同的数据对象有不同的存取权限
 - 不同的用户对同一对象也有不同的权限
 - 用户还可将其拥有的存取权限转授给其他用户

自主存取控制方法 1

- 通过 SQL 的GRANT 语句和REVOKE 语句实现
- 用户权限组成
 - ◆ 数据库对象
 - ◆ 操作类型
- 定义存取权限称为授权
 - 定义用户存取权限：定义用户可以在哪些数据库对象上进行哪些类型的操作

自主存取控制方法 2

● 关系数据库系统中存取控制对象

对象类型	对象	操 作 类 型
数据库 模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表 和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

SQL中的授权机制

- 数据库管理员：
 - 拥有所有对象的所有权限
 - 根据实际情况不同的权限授予不同的用户
- 用户：
 - 拥有自己建立的对象的全部的操作权限
 - 可以使用GRANT，把权限授予其他用户
- 被授权的用户
 - 如果具有“继续授权”的许可，可以把获得的权限再授予其他用户
- 所有授予出去的权力在必要时又都可用REVOKE语句收回



GRANT 1

- GRANT语句的一般格式:

GRANT <权限>[,<权限>]...

ON <对象类型> <对象名>[,<对象类型> <对象名>]...

TO <用户>[,<用户>]...

[WITH GRANT OPTION];

- 语义：将对指定操作对象的指定操作权限授予指定的用户

- 发出GRANT:

- 数据库管理员
- 数据库对象创建者（即属主Owner）
- 拥有该权限的用户

- 按受权限的用户

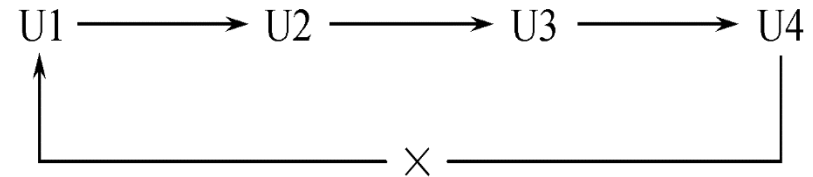
- 一个或多个具体用户
- PUBLIC（即全体用户）

GRANT 2

- WITH GRANT OPTION子句:

- 指定: 可以再授予
- 没有指定: 不能传播

- 不允许循环授权



- [例4.1] 把查询Student表权限授给用户U1

```
GRANT SELECT
ON TABLE Student
TO U1;
```

- [例4.2] 把对Student表和Course表的全部权限授予用户U2和U3

```
GRANT ALL PRIVILIGES
ON TABLE Student, Course
TO U2, U3;
```

GRANT 3

- [例4.3] 把对表SC的查询权限授予所有用户

GRANT SELECT

ON TABLE SC

TO PUBLIC;

- [例4.4] 把查询Student表和修改学生学号的权限授给用户U4

GRANT UPDATE(Sno), SELECT

ON TABLE Student

TO U4;

- 对属性列的授权时必须明确指出相应属性列名

- [例4.5] 把对表SC的INSERT权限授予U5用户，并允许他再将此权限授予其他用户

GRANT INSERT

ON TABLE SC

TO U5

WITH GRANT OPTION;

- 执行例4.5后，U5不仅拥有了对表SC的INSERT权限，还可以传播此权限：

- [例4.6] GRANT INSERT

ON TABLE SC

TO U6

WITH GRANT OPTION;

- [例4.7]同样，U6还可以将此权限授予U7，但U7不能再传播此权限。

GRANT INSERT

ON TABLE SC

TO U7;

传播权限

- 执行了例4.1~例4.7语句后学生-课程数据库中的用户权限定义表

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系Student	SELECT	不能
DBA	U2	关系Student	ALL	不能
DBA	U2	关系Course	ALL	不能
DBA	U3	关系Student	ALL	不能
DBA	U3	关系Course	ALL	不能
DBA	PUBLIC	关系SC	SELECT	不能
DBA	U4	关系Student	SELECT	不能
DBA	U4	属性列Student.Sno	UPDATE	不能
DBA	U5	关系SC	INSERT	能
U5	U6	关系SC	INSERT	能
U6	U7	关系SC	INSERT	不能

REVOKE 1

- REVOKE
- 授予的权限可以由数据库管理员或其他授权者用 REVOKE 语句收回
- REVOKE 语句的一般格式为：

REVOKE <权限>[,<权限>]...

ON <对象类型> <对象名>[,<对象类型><对象名>]...

FROM <用户>[,<用户>]...[CASCADE | RESTRICT];

REVOKE 2

- [例4.8] 把用户U4修改学生学号的权限收回

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

- [例4.9] 收回所有用户对表SC的查询权限

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

- [例4.10] 把用户U5对SC表的INSERT权限收回

```
REVOKE INSERT
ON TABLE SC
FROM U5 CASCADE ;
```

- 将用户U5的INSERT权限收回的时候使用CASCADE, 则同时收回U6或U7的INSERT权限, 否则拒绝执行该语句
- 如果U6或U7还从其他用户处获得对SC表的INSERT权限, 则他们仍具有此权限, 系统只收回直接或间接从U5处获得的权限

REVOKE 3

- 执行例4.8~4.10语句后学生-课程数据库中的用户权限定义表

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系Student	SELECT	不能
DBA	U2	关系Student	ALL	不能
DBA	U2	关系Course	ALL	不能
DBA	U3	关系Student	ALL	不能
DBA	U3	关系Course	ALL	不能
DBA	U4	关系Student	SELECT	不能

创建数据库模式的权限 1

- 数据库管理员在创建用户时实现
 - CREATE USER语句格式
 - CREATE USER <username>
 - [WITH][DBA|RESOURCE|CONNECT];
- 注：CREATE USER不是SQL标准，各个系统的实现相差甚远
- 只有系统的超级用户才有权创建一个新的数据库用户
- 新创建的数据库用户有三种权限：
CONNECT、RESOURCE和DBA
 - 如没有指定创建的新用户的权限，默认该用户拥有CONNECT权限。拥有CONNECT权限的用户不能创建新用户，不能创建模式，也不能创建基本表，只能登录数据库
 - 拥有RESOURCE权限的用户能创建基本表和视图，成为所创建对象的属主。但不能创建模式，不能创建新的用户
 - 拥有DBA权限的用户是系统中的超级用户，可以创建新的用户、创建模式、创建基本表和视图等；DBA拥有对所有数据库对象的存取权限，还可以把这些权限授予一般用户

创建数据库模式的权限 2

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库， 执行数据查询和 操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	不可以	不可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

数据库角色 1

- 数据库角色：被命名的一组与数据库操作相关的权限

- 角色是权限的集合
- 可以为一组具有相同权限的用户创建一个角色
- 简化授权的过程

- 角色的创建

```
CREATE ROLE <角色名>
```

- 给角色授权

```
GRANT <权限>[,<权限>]...
```

```
ON <对象类型>对象名  
TO <角色>[,<角色>]...
```



数据库角色 2

- 将一个角色授予其他的角色或用户
GRANT <角色1>[,<角色2>]...
TO <角色3>[,<用户1>]...
[WITH ADMIN OPTION]
 - 该语句把角色授予某用户，或授予另一个角色
 - 授予者是角色的创建者或拥有在这个角色上的ADMIN OPTION
 - 指定了WITH ADMIN OPTION则获得某种权限的角色或用户还可以把这种权限授予其他角色
- 一个角色的权限：直接授予这个角色的全部权限加上其他角色授予这个角色的全部权限
- 角色权限的收回
REVOKE <权限>[,<权限>]...
ON <对象类型> <对象名>
FROM <角色>[,<角色>]...
- 用户可以回收角色的权限，从而修改角色拥有的权限
- REVOKE执行者是
 - 角色的创建者
 - 拥有在这个（些）角色上的ADMIN OPTION

数据库角色 3

- [例4.11] 通过角色来实现将一组权限授予一个用户。步骤如下：

- 首先创建一个角色 R1

```
CREATE ROLE R1;
```

- 然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

```
GRANT SELECT, UPDATE, INSERT
```

```
ON TABLE Student
```

```
TO R1;
```

- 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

```
GRANT R1
```

```
TO 王平,张明,赵玲;
```

- 可以一次性通过R1来回收王平的这3个权限

```
REVOKE R1
```

```
FROM 王平;
```



数据库角色 4

- [例4.12] 角色的权限修改

GRANT DELETE

ON TABLE Student

TO R1;

- 使角色R1在原来的基础上增加了Student表的DELETE 权限

- [例4.13] 使R1减少了SELECT权限

REVOKE SELECT

ON TABLE Student

FROM R1;



自主存取控制缺点

- 可能存在数据的“无意泄露”
 - 原因：这种机制仅仅通过对数据的存取权限来进行安全控制，而数据本身并无安全性标记
 - 解决：对系统控制下的所有主客体实施强制存取控制策略



强制存取控制

- 强制存取控制 (Mandatory Access Control, 简称 MAC)
 - B1级, 保证更高层次的安全性
 - 每一个数据对象被标以一定的密级
 - 每一个用户也被授予某一个级别的许可证
 - 对于任意一个对象, 只有具有合法许可证的用户才可以存取
 - 用户不能直接感知或进行控制
 - 适用于对数据有严格而固定密级分类的部门
 - ◆ 军事部门
 - ◆ 政府部门

强制存取控制方法-实体

- 在强制存取控制中，数据库管理系统所管理的全部实体被分为主体和客体两大类
 - 主体是系统中的活动实体
 - ◆ 数据库管理系统所管理的实际用户
 - ◆ 代表用户的各进程
 - 客体是系统中的被动实体，受主体操纵
 - ◆ 文件、基本表、索引、视图



强制存取控制方法-敏感度标记

- 敏感度标记 (Label)
 - 对于主体和客体, DBMS为它们每个实例 (值) 指派一个敏感度标记 (Label)
 - 敏感度标记分成若干级别
 - ◆ 绝密 (Top Secret, TS)
 - ◆ 机密 (Secret, S)
 - ◆ 可信 (Confidential, C)
 - ◆ 公开 (Public, P)
 - ◆ $TS \geq S \geq C \geq P$
- 主体的敏感度标记称为许可证级别 (Clearance Level)
- 客体的敏感度标记称为密级 (Classification Level)



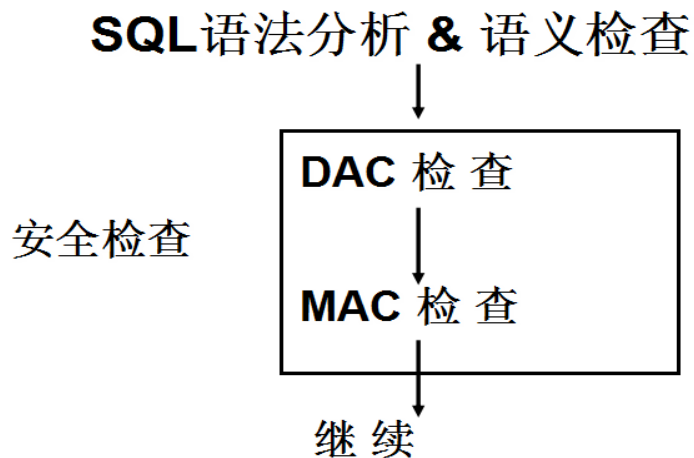
强制存取控制方法-强制存取控制规则

- 强制存取控制规则
 - 仅当主体的许可证级别大于或等于客体的密级时，该主体才能读取相应的客体
 - 仅当主体的许可证级别小于或等于客体的密级时，该主体才能写相应的客体
- 强制存取控制（MAC）是对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据。



DAC + MAC

- 实现强制存取控制时要首先实现自主存取控制
 - 原因：较高安全性级别提供的安全保护要包含较低级别的所有保护
- 自主存取控制与强制存取控制共同构成数据库管理系统的安全机制，先进行自主存取控制检查，通过自主存取控制检查的数据对象再由系统进行强制存取控制检查，只有通过强制存取控制检查的数据对象方可存取。





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch25 视图机制、审计、数据加密及其他

Software Institute
Nanjing University
Bei Jia

视图机制 1

- 把要保密的数据对无权存取这些数据的用户隐藏起来，对数据提供一定程度的安全保护
- 间接地实现支持存取谓词的用户权限定义



视图机制 2

- [例4.14] 建立计算机系学生的视图，把对该视图的 SELECT 权限授于王平，把该视图上的所有操作权限授于张明

- 先建立计算机系学生的视图

CS_Student

```
CREATE VIEW CS_Student
```

```
AS
```

```
SELECT *
```

```
FROM Student
```

```
WHERE Sdept='CS';
```

- 在视图上进一步定义存取权限

```
GRANT SELECT
```

```
ON CS_Student
```

```
TO 王平;
```

```
GRANT ALL PRIVILIGES
```

```
ON CS_Student
```

```
TO 张明;
```

审计

- 审计

- 启用一个专用的审计日志 (Audit Log) 将用户对数据库的所有操作记录在上面
- 审计员利用审计日志监控数据库中的各种行为, 找出非法存取数据的人、时间和内容
- C2以上安全级别的DBMS必须具有审计功能

- 审计功能的可选性

- 审计很费时间和空间
- DBA可以根据应用对安全性的要求, 灵活地打开或关闭审计功能
- 审计功能主要用于安全性要求较高的部门



审计事件

- 服务器事件

- ◆ 审计数据库服务器发生的事件

- 系统权限

- ◆ 对系统拥有的结构或模式对象进行操作的审计
 - ◆ 要求该操作的权限是通过系统权限获得的

- 语句事件

- ◆ 对SQL语句，如DDL、DML、DQL及DCL语句的审计

- 模式对象事件

- ◆ 对特定模式对象上进行的SELECT或DML操作的审计



审计功能

- 基本功能
 - 提供多种审计查阅方式
- 多套审计规则：一般在初始化设定
- 提供审计分析和报表功能
- 审计日志管理功能
 - 防止审计员误删审计记录，审计日志必须先转储后删除
 - 对转储的审计记录文件提供完整性和保密性保护
 - 只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等
- 提供查询审计设置及审计记录信息的专门视图

审计级别

- 用户级审计

- 任何用户可设置的审计
- 主要是用户针对自己创建的数据库表和视图进行审计

- 系统级审计

- 只能由数据库管理员设置
- 监测成功或失败的登录要求、监测授权和收回操作以及其他数据库级权限下的操作

审计语句

- AUDIT语句和NOAUDIT语句
 - AUDIT语句：设置审计功能
 - NOAUDIT语句：取消审计功能
- [例4.15] 对修改SC表结构或修改SC表数据的操作进行审计

```
AUDIT ALTER,UPDATE  
ON SC;
```

- [例4.16] 取消对SC表的一切审计

```
NOAUDIT ALTER,UPDATE  
ON SC;
```



数据加密

- 数据加密

- 防止数据库中数据在存储和传输中失密的有效手段

- 加密的基本思想

- 根据一定的算法将原始数据—明文 (Plain text) 变换为不可直接识别的格式—密文 (Cipher text)

- 加密方法

- 存储加密
 - 传输加密



存储加密

● 透明存储加密

- 内核级加密保护方式，对用户完全透明
- 将数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密
- 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可
- 内核级加密方法：性能较好，安全完备性较高

● 非透明存储加密

- 通过多个加密函数实现

传输加密

- 链路加密

- 在链路层进行加密
- 传输信息由报头和报文两部分组成
- 报文和报头均加密

- 端到端加密

- 在发送端加密，接收端解密
- 只加密报文不加密报头
- 所需密码设备数量相对较少，容易被非法监听者发现并从中获取敏感信息



其他安全性保护

- 推理控制

- 避免用户利用能够访问的数据推知更高密级的数据

- 隐蔽信道

- 间接数据传递

- 数据隐私保护

- 描述个人控制其不愿他人知道或他人不便知道的个人数据的能力
- 范围很广：数据收集、数据存储、数据发布和数据发布等各个阶段





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch26 数据库完整性

Software Institute
Nanjing University
Bei Jia

数据库完整性

- 数据库的完整性

- 数据的正确性

- ◆ 是指数据是符合现实世界语义，反映了当前实际状况的

- 数据的相容性

- ◆ 是指数据库同一对象在不同关系表中的数据是符合逻辑的

- 例如，

- ◆ 学生的学号必须唯一
 - ◆ 性别只能是男或女
 - ◆ 本科学生年龄的取值范围为14~50的整数
 - ◆ 学生所选的课程必须是学校开设的课程，学生所在的院系必须是学校已成立的院系
 - ◆



完整性 vs. 安全性

● 数据的完整性和安全性是两个不同概念

■ 数据的完整性

- 防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据
- 防范对象：不合语义的、不正确的数据

■ 数据的安全性

- 保护数据库，防止恶意的破坏和非法的存取
- 防范对象：非法用户和非法操作



完整性机制

- 提供定义完整性约束条件的机制

- 完整性约束条件也称为完整性规则，是数据库中的数据必须满足的语义约束条件
- SQL标准使用了一系列概念来描述完整性，包括关系模型的实体完整性、参照完整性和用户定义完整性
- 这些完整性一般由SQL的数据定义语言语句来实现

- 提供完整性检查的方法

- 数据库管理系统中检查数据是否满足完整性约束条件的机制称为完整性检查。
- 一般在INSERT、UPDATE、DELETE语句执行后开始检查，也可以在事务提交时检查

- 违约处理

- 数据库管理系统若发现用户的操作违背了完整性约束条件，就采取一定的动作
 - ◆ 拒绝 (NO ACTION) 执行该操作
 - ◆ 级连 (CASCADE) 执行其他操作





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch27 实体完整性

Software Institute
Nanjing University
Bei Jia

实体完整性定义 1

- 关系模型的实体完整性
 - CREATE TABLE中用PRIMARY KEY定义
- 单属性构成的码有两种说明方法
 - 定义为列级约束条件
 - 定义为表级约束条件
- 对多个属性构成的码只有一种说明方法
 - 定义为表级约束条件

实体完整性定义 2

● [例5.1] 将Student表中的Sno属性定义为码

■ 在列级定义主码

```
CREATE TABLE Student
( Sno CHAR(9) PRIMARY
KEY,
  Sname CHAR(20) NOT
NULL,
  Ssex CHAR(2),
  Sage SMALLINT,
  Sdept CHAR(20)
);
```

■ 在表级定义主码

```
CREATE TABLE Student
( Sno CHAR(9),
  Sname CHAR(20)
NOT NULL,
  Ssex CHAR(2),
  Sage SMALLINT,
  Sdept CHAR(20),
  PRIMARY KEY (Sno)
);
```



实体完整性定义 3

- [例5.2] 将SC表中的Sno, Cno属性组定义为码

```
CREATE TABLE SC
```

```
( Sno CHAR(9) NOT NULL,
```

```
  Cno CHAR(4) NOT NULL,
```

```
  Grade SMALLINT,
```

```
  PRIMARY KEY (Sno,Cno)  /*只能在表级定义主码*/
```

```
);
```



实体完整性检查和违约处理

- 插入或对主码列进行更新操作时，关系数据库管理系统按照实体完整性规则自动进行检查。包括：
 - 检查主码值是否唯一，如果不唯一则拒绝插入或修改
 - 检查主码的各个属性是否为空，只要有一个为空就拒绝插入或修改



实体完整性检查 1

- 检查记录中主码值是否唯一的一种方法是进行全表扫描
 - 依次判断表中每一条记录的主码值与将插入记录上的主码值（或者修改的新主码值）是否相同
 - 十分耗时

待插入记录

Key _i	F2 _i	F3 _i	F4 _i	F5 _i
------------------	-----------------	-----------------	-----------------	-----------------

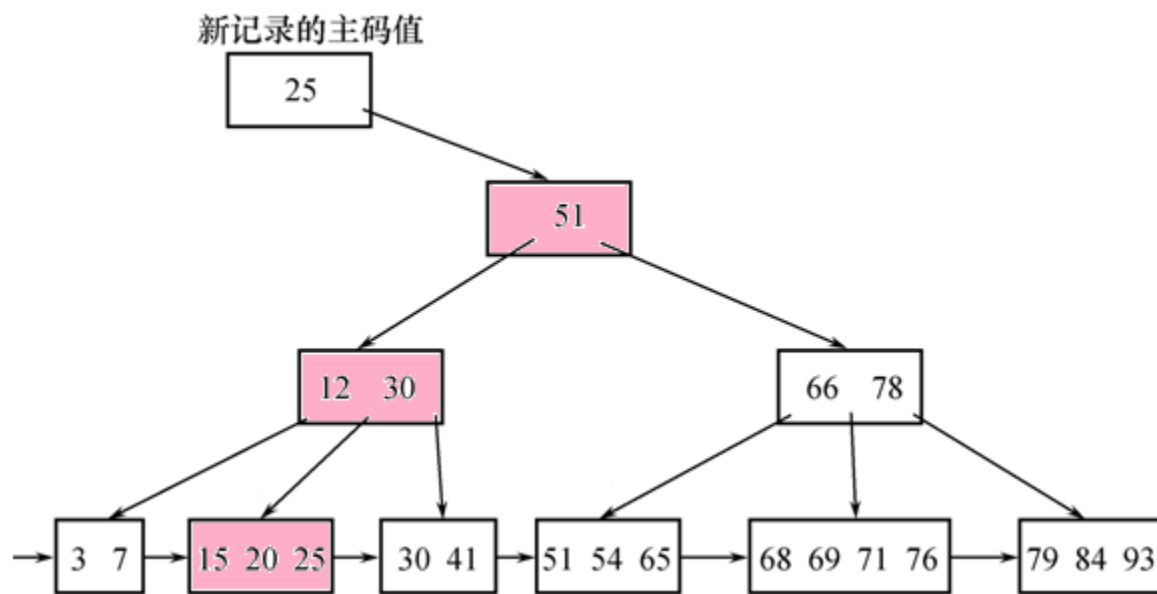
基本表

Key1	F21	F31	F41	F51
Key2	F22	F32	F42	F52
Key3	F23	F33	F43	F53
⋮				



实体完整性检查 2

- 为避免对基本表进行全表扫描，RDBMS核心一般都在主码上自动建立一个索引，如B+树索引
 - 新插入记录的主码值是25
 - 通过主码索引，从B+树的根结点开始查找
 - 读取3个结点：根结点（51）、中间结点（12 30）、叶结点（15 20 25）
 - 该主码值已经存在，不能插入这条记录





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch28 参照完整性

Software Institute
Nanjing University
Bei Jia

参照完整性定义

- 关系模型的参照完整性定义
 - 在CREATE TABLE中用FOREIGN KEY短语定义哪些列为外码
 - 用REFERENCES短语指明这些外码参照哪些表的主码
- [例5.3]定义SC中的参照完整性

```
CREATE TABLE SC
```

```
( Sno CHAR(9) NOT NULL,
```

```
  Cno CHAR(4) NOT NULL,
```

```
  Grade SMALLINT,
```

```
  PRIMARY KEY (Sno, Cno), /*在表级定义实体完整性*/
```

```
  FOREIGN KEY (Sno) REFERENCES Student(Sno),
```

```
  /*在表级定义参照完整性*/
```

```
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
```

```
  /*在表级定义参照完整性*/
```

```
);
```



参照完整性检查 1

- 一个参照完整性将两个表中的相应元组联系起来
- 对被参照表和参照表进行增删改操作时有可能破坏参照完整性，必须进行检查
- 例如，对表SC和Student有四种可能破坏参照完整性的情况：
 - SC表中增加一个元组，该元组的Sno属性的值在表Student中找不到一个元组，其Sno属性的值与之相等。
 - 修改SC表中的一个元组，修改后该元组的Sno属性的值在表Student中找不到一个元组，其Sno属性的值与之相等。
 - 从Student表中删除一个元组，造成SC表中某些元组的Sno属性的值在表Student中找不到一个元组，其Sno属性的值与之相等。
 - 修改Student表中一个元组的Sno属性，造成SC表中某些元组的Sno属性的值在表Student中找不到一个元组，其Sno属性的值与之相等。



参照完整性检查 2

表5.1 可能破坏参照完整性的情况及违约处理

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外码值	拒绝
删除元组 →	可能破坏参照完整性	拒绝/级连删除/设置为空值
修改主码值 →	可能破坏参照完整性	拒绝/级连修改/设置为空值



参照完整性违约处理 1

- 参照完整性违约处理

- 拒绝 (NO ACTION) 执行

- ◆ 不允许该操作执行。该策略一般设置为默认策略

- 级联 (CASCADE) 操作

- ◆ 当删除或修改被参照表 (Student) 的一个元组造成了与参照表 (SC) 的不一致, 则删除或修改参照表中的所有造成不一致的元组

- 设置为空值 (SET-NULL)

- ◆ 当删除或修改被参照表的一个元组时造成了不一致, 则将参照表中的所有造成不一致的元组的对应属性设置为空值。
- ◆ 对于参照完整性, 除了应该定义外码, 还应定义外码列是否允许空值



参照完整性违约处理 2

- 例如，有下面2个关系

- 学生（学号，姓名，性别，专业号，年龄）
 - 专业（专业号，专业名）
- ◆ 假设专业表中某个元组被删除，专业号为12
 - ◆ 按照设置为空值的策略，就要把学生表中专业号=12的所有元组的专业号设置为空值
 - ◆ 对应语义：某个专业删除了，该专业的所有学生专业未定，等待重新分配专业

外码



参照完整性违约处理 3

- [例5.4] 显式说明参照完整性的违约处理示例

```
CREATE TABLE SC
```

```
( Sno CHAR(9) NOT NULL,
```

```
  Cno CHAR(4) NOT NULL,
```

```
  Grade SMALLINT,
```

```
  PRIMARY KEY(Sno,Cno),
```

```
  FOREIGN KEY (Sno) REFERENCES Student(Sno)
```

```
    ON DELETE CASCADE      /*级联删除SC表中相应的元组*/
```

```
    ON UPDATE CASCADE,     /*级联更新SC表中相应的元组*/
```

```
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
```

```
    ON DELETE NO ACTION
```

```
    /*当删除course 表中的元组造成了与SC表不一致时拒绝删除*/
```

```
    ON UPDATE CASCADE
```

```
    /*当更新course表中的cno时，级联更新SC表中相应的元组*/
```

```
);
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch29 用户定义的完整性

Software Institute
Nanjing University
Bei Jia

用户定义的完整性

- 用户定义的完整性是：针对某一具体应用的数据必须满足的语义要求
 - 属性上的约束条件
 - 元组上的约束条件
 - ◆ 同属性值限制相比，元组级的限制可以设置不同属性之间的取值的相互约束条件
- 关系数据库管理系统提供了定义和检验用户定义完整性的机制，不必由应用程序承担
 - 插入元组或修改属性的值时，关系数据库管理系统检查约束条件是否被满足
 - 如果不满足则操作被拒绝执行



属性上约束条件的定义- 列值非空

- [例5.5] 在定义SC表时, 说明Sno、Cno、Grade属性不允许取空值。

```
CREATE TABLE SC
```

```
( Sno CHAR(9) NOT NULL,
```

```
  Cno CHAR(4) NOT NULL,
```

```
  Grade SMALLINT NOT NULL,
```

```
  PRIMARY KEY (Sno, Cno),
```

```
  ...
```

/* 如果在表级定义实体完整性, 隐含了Sno, Cno不允许取空值, 则在
列级不允许取空值的定义 可以不写 */

```
);
```



属性上约束条件的定义- 列值唯一

- [例5.6]建立部门表DEPT，要求部门名称Dname列取值唯一，部门编号Deptno列为主码

```
CREATE TABLE DEPT
```

```
( Deptno NUMERIC(2),
```

```
  Dname CHAR(9) UNIQUE NOT NULL,
```

```
  /*要求Dname列值唯一，并且不能取空值*/
```

```
  Location CHAR(10),
```

```
  PRIMARY KEY (Deptno)
```

```
);
```



属性上约束条件的定义- 列值是否满足条件表达式 1

- 用CHECK短语指定列值应该满足的条件
- [例5.7] Student表的Ssex只允许取“男”或“女”。

```
CREATE TABLE Student
```

```
( Sno CHAR(9) PRIMARY KEY,
```

```
  Sname CHAR(8) NOT NULL,
```

```
  Ssex CHAR(2) CHECK (Ssex IN ('男','女')) ,
```

```
/*性别属性Ssex只允许取'男'或'女'*/
```

```
  Sage SMALLINT,
```

```
  Sdept CHAR(20)
```

```
);
```



属性上约束条件的定义- 列值是否满足条件表达式 2

- [例5.8] SC表的Grade的值应该在0和100之间。

```
CREATE TABLE SC
```

```
( Sno CHAR(9),
```

```
  Cno CHAR(4),
```

```
  Grade SMALLINT CHECK (Grade>=0 AND Grade <=100),
```

```
    /*Grade取值范围是0到100*/
```

```
  PRIMARY KEY (Sno,Cno),
```

```
  FOREIGN KEY (Sno) REFERENCES Student(Sno),
```

```
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
```

```
);
```

元组上约束条件的定义

- [例5.9]当学生的性别是男时，其名字不能以Ms.打头。

```
CREATE TABLE Student
```

```
( Sno   CHAR(9),
```

```
  Sname CHAR(8) NOT NULL,
```

```
  Ssex   CHAR(2),
```

```
  Sage   SMALLINT,
```

```
  Sdept  CHAR(20),
```

```
  PRIMARY KEY (Sno),
```

```
  CHECK (Ssex='女' OR Sname NOT LIKE 'Ms.%')
```

```
    /*定义了元组中Sname和 Ssex两个属性值之间的约束条件*/
```

```
);
```

- 性别是女性的元组都能通过该项检查，因为Ssex='女'成立;
- 当性别是男性时，要通过检查则名字一定不能以Ms.打头



完整性约束命名子句

- 创建完整性约束

- 完整性约束命名子句

- ◆ CONSTRAINT <完整性约束条件名><完整性约束条件>
 - ◆ <完整性约束条件>包括NOT NULL、UNIQUE、PRIMARY KEY短语、FOREIGN KEY短语、CHECK短语等

- 修改完整性约束

- 使用ALTER TABLE语句修改表中的完整性限制



创建完整性约束 1

- [例5.10]建立学生登记表Student，要求学号在90000~99999之间，姓名不能取空值，年龄小于30，性别只能是“男”或“女”。

```
CREATE TABLE Student
```

```
( Sno NUMERIC(6)
```

```
  CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),
```

```
  Sname CHAR(20)
```

```
  CONSTRAINT C2 NOT NULL,
```

```
  Sage NUMERIC(3)
```

```
  CONSTRAINT C3 CHECK (Sage < 30),
```

```
  Ssex CHAR(2)
```

```
  CONSTRAINT C4 CHECK (Ssex IN ( '男','女')),
```

```
  CONSTRAINT StudentKey PRIMARY KEY(Sno)
```

```
);
```

- 在Student表上建立了5个约束条件，包括主码约束（命名为StudentKey）以及C1、C2、C3、C4四个列级约束。



创建完整性约束 2

- [例5.11]建立教师表TEACHER，要求每个教师的应发工资不低于3000元。应发工资是工资列Sal与扣除项Deduct之和。

```
CREATE TABLE TEACHER
```

```
(  Eno  NUMERIC(4) PRIMARY KEY  /*在列级定义主码*/
```

```
   Ename CHAR(10),
```

```
   Job   CHAR(8),
```

```
       Sal   NUMERIC(7,2),
```

```
       Deduct NUMERIC(7,2),
```

```
       Deptno NUMERIC(2),
```

```
       CONSTRAINT TEACHERFKKey FOREIGN KEY (Deptno)  
REFERENCES DEPT(Deptno),
```

```
       CONSTRAINT C1 CHECK (Sal + Deduct >= 3000)
```

```
);
```



修改完整性约束

- [例5.12]去掉例5.10 Student表中对性别的限制。

```
ALTER TABLE Student
```

```
DROP CONSTRAINT C4;
```

- [例5.13] 修改表Student中的约束条件，要求学号改为在900000~999999之间，年龄由小于30改为小于40

- 可以先删除原来的约束条件，再增加新的约束条件

```
ALTER TABLE Student
```

```
DROP CONSTRAINT C1;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999),
```

```
ALTER TABLE Student
```

```
DROP CONSTRAINT C3;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C3 CHECK(Sage < 40);
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch30 断言

Software Institute
Nanjing University
Bei Jia

断言

- SQL中，可以使用 CREATE ASSERTION语句，通过声明性断言来指定更具一般性的约束。
- 可以定义涉及多个表的或聚集操作的比较复杂的完整性约束。
- 断言创建以后，任何对断言中所涉及的关系的操作都会触发关系数据库管理系统对断言的检查，任何使断言不为真值的操作都会被拒绝执行



断言相关语句

- 创建断言的语句格式

- CREATE ASSERTION<断言名><CHECK 子句>
- 每个断言都被赋予一个名字，<CHECK 子句>中的约束条件与WHERE子句的条件表达式类似。

- 删除断言的语句格式为

- DROP ASSERTION <断言名>;
- 如果断言很复杂，则系统在检测和维护断言的开销较高，这是在使用断言时应该注意的

创建断言 1

- [例5.18] 限制数据库课程最多60名学生选修

```
CREATE ASSERTION ASSE_SC_DB_NUM
```

```
CHECK (60 >= (select count(*)
```

```
/*此断言的谓词涉及聚集操作count的SQL语句*/
```

```
From Course,SC
```

```
Where SC.Cno=Course.Cno and
```

```
Course.Cname ='数据库')
```

```
);
```

创建断言 2

- [例5.19]限制每一门课程最多60名学生选修

```
CREATE ASSERTION ASSE_SC_CNUM1  
CHECK(60 >= ALL (SELECT count(*)  
                  FROM      SC  
                  GROUP by cno)  
);
```

/*此断言的谓词，涉及聚集操作count 和分组函数
group by的SQL语句*/



创建断言 3

- [例5.20]限制每个学期每一门课程最多60名学生选修。首先需要修改SC表的模式，增加一个“学期 (TERM)”属性

```
ALTER TABLE SC ADD TERM DATE;
```

- 然后，定义断言：

```
CREATE ASSERTION ASSE_SC_CNUM2  
CHECK(60 >= ALL (SELECT count(*)  
FROM SC  
GROUP by cno,TERM)  
);
```





南京大学软件学院
NANJING UNIVERSITY · SOFTWARE INSTITUTE



数据管理基础

ch31 触发器

Software Institute
Nanjing University
Bei Jia

触发器

- 触发器（Trigger）是用户定义在关系表上的一类由事件驱动的特殊过程
 - 触发器保存在数据库服务器中
 - 任何用户对表的增、删、改操作均由服务器自动激活相应的触发器
 - 触发器可以实施更为复杂的检查和操作，具有更精细和更强大的数据控制能力
- 不同的RDBMS产品触发器语法各不相同



定义触发器 1

- CREATE TRIGGER语法格式

CREATE TRIGGER <触发器名>

{BEFORE | AFTER} <触发事件> ON <表名>

REFERENCING NEW|OLD ROW AS<变量>

FOR EACH {ROW | STATEMENT}

[WHEN <触发条件>]<触发动作体>

- 触发器又叫做事件-条件-动作（event-condition-action）规则。

- 当特定的系统事件发生时，对规则的条件进行检查，如果条件成立则执行规则中的动作，否则不执行该动作。规则中的动作体可以很复杂，通常是一段SQL存储过程。



定义触发器 2

- 表的拥有者才可以在表上创建触发器
- 触发器名
 - 触发器名可以包含模式名，也可以不包含模式名
 - 同一模式下，触发器名必须是唯一的
 - 触发器名和表名必须在同一模式下
- 表名
 - 触发器只能定义在基本表上，不能定义在视图上
 - 当基本表的数据发生变化时，将激活定义在该表上相应触发事件的触发器
- 触发事件
 - 触发事件可以是INSERT、DELETE或UPDATE，也可以是这几个事件的组合
 - 还可以UPDATE OF<触发列，...>，即进一步指明修改哪些列时激活触发器
 - AFTER/BEFORE是触发的时机
 - ◆ AFTER表示在触发事件的操作执行之后激活触发器
 - ◆ BEFORE表示在触发事件的操作执行之前激活触发器
- 触发器类型
 - 行级触发器 (FOR EACH ROW)
 - 语句级触发器 (FOR EACH STATEMENT)
- 触发条件
 - 触发器被激活时，只有当触发条件为真时触发动作体才执行;否则触发动作体不执行。
 - 如果省略WHEN触发条件，则触发动作体在触发器激活后立即执行
- 触发动作体
 - 触发动作体可以是一个匿名PL/SQL过程块
 - 也可以是对已创建存储过程的调用
 - 如果是行级触发器，用户都可以在过程体中使用NEW和OLD引用事件之后的新值和事件之前的旧值
 - 如果是语句级触发器，则不能在触发动作体中使用NEW或OLD进行引用
 - 如果触发动作体执行失败，激活触发器的事件就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生任何变化



触发器类型

- 例如,在例5.11的TEACHER表上创建一个 AFTER UPDATE触发器, 触发事件是 UPDATE语句:
 - UPDATE TEACHER SET Deptno=5;
 - 假设表TEACHER有1000行
 - ◆ 如果是语句级触发器, 那么执行完该语句后, 触发动作只发生一次
 - ◆ 如果是行级触发器, 触发动作将执行1000次



触发器示例 1

- [例5.21]当对表SC的Grade属性进行修改时，若分数增加了10%则将此次操作记录到下面表中：SC_U (Sno,Cno,Oldgrade,Newgrade) 其中Oldgrade是修改前的分数，Newgrade是修改后的分数。

```
CREATE TRIGGER SC_T
AFTER UPDATE OF Grade ON SC
REFERENCING
    OLD row AS OldTuple,
    NEW row AS NewTuple
FOR EACH ROW
WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)
    INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)
VALUES(OldTuple.Sno,OldTuple.Cno,OldTuple.Grade,NewTuple.Grade)
```



触发器示例 2

- [例5.22] 将每次对表Student的插入操作所增加的学生个数记录到表StudentInsertLog中。

```
CREATE TRIGGER Student_Count
```

```
AFTER INSERT ON Student
```

```
    /*指明触发器激活的时间是在执行INSERT后*/
```

```
    REFERENCING
```

```
    NEW TABLE AS DELTA
```

```
    FOR EACH STATEMENT
```

```
/*语句级触发器, 即执行完INSERT语句后下面的触发动作体才执行一次*/
```

```
    INSERT INTO StudentInsertLog (Numbers)
```

```
    SELECT COUNT(*) FROM DELTA
```



触发器示例 3

- [例5.23] 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”。

```
CREATE TRIGGER Insert_Or_Update_Sal
BEFORE INSERT OR UPDATE ON Teacher
    /*触发事件是插入或更新操作*/
FOR EACH ROW    /*行级触发器*/
BEGIN
    /*定义触发动作体，是PL/SQL过程块*/
    IF (new.Job='教授') AND (new.Sal < 4000)
    THEN new.Sal :=4000;
    END IF;
END;
```



激活触发器

- 触发器的执行，是由触发事件激活的，并由数据库服务器自动执行
- 一个数据表上可能定义了多个触发器，遵循如下的执行顺序：
 - ◆ 执行该表上的BEFORE触发器；
 - ◆ 激活触发器的SQL语句；
 - ◆ 执行该表上的AFTER触发器。



删除触发器

- 删除触发器的SQL语法:

DROP TRIGGER <触发器名> ON <表名>;

- 触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除。

